

Response to Reviewers

A PROBABILISTIC CHOREOGRAPHY LANGUAGE FOR PRISM

Marco Carbone and Adele Veschetti

General Response

We thank the Editor and the Reviewers for their detailed and constructive feedback. We have revised the paper accordingly and believe that the changes substantially improve both clarity and technical depth.

One major change in this revision concerns the definition of projection. In the previous version, the projection relied on explicit counters (natural numbers, denoted by ι) to generate fresh identifiers; this mechanism was cumbersome and could be error-prone. We now adopt a more direct approach by annotating choreographies with unique labels, which the projection can use directly. This yields a cleaner and more easily readable projection in the theoretical development. For practical reasons, our implementation still uses counters; these can be seen as a concrete realisation of the label-generation mechanism that the paper now abstracts into the choreography annotations.

Below we address each reviewer point by point. Minor comments have been omitted from the reply, but addressed.

Reviewer 1

Reviewer comment. I found the technical result establishing operational equivalence between choreographies and their projections rather weak. [...] This may ultimately be irrelevant, but I believe the issue should be discussed in the paper. I would also expect a result establishing operational correspondence for a sequence of reductions.

Response. We thank the reviewer for highlighting the mismatch between standard operational-correspondence results for choreographies and this paper focussing on PRISM. PRISM modules are declarative (a fixed set of commands whose enabling depends on the current state), whereas choreographies have an explicit control-flow syntax; therefore, in the projection, control-flow is reified into the (extended) runtime state via the variables s_p , yielding an extended state space S_+ . As a consequence, reductions of the choreography correspond to transitions on S_+ in the projected model rather than to syntactic evolution of PRISM modules. In the revised manuscript, we make this design choice explicit and discuss the resulting asymmetry (projection-after-reduction may yield fewer commands than reduction-after-projection). At the end of Section 5, we include a concrete example illustrating that, while the set of commands may differ, the projected models exhibit the same observable behaviour up to a suitable PRISM-induced equivalence; we leave the identification of the most appropriate equivalence (e.g., probabilistic bisimulation) and the corresponding strengthening of the theorem to future work.

Changes in the manuscript. We expanded the discussion around Theorem 1 to clarify (i) why standard reduction-to-reduction correspondence does not directly apply to PRISM, (ii) how control-flow information is encoded in the extended state S_+ via the distributed state variables, and (iii) why the correspondence statement is formulated in terms of transitions on S_+ rather than syntactic changes of modules. We also added a detailed remark and worked example to make the asymmetry and its implications explicit, and we added a paragraph outlining how a future extension based on probabilistic bisimulation could capture correspondence across different command sets.

Reviewer comment. Furthermore, the proof of this correspondence (Theorem 1) reads more like a proof sketch than a complete proof; a more detailed and carefully paced proof would be preferable.

Response. We thank the reviewer for this suggestion. In the revised version, we have expanded the proof of Theorem 1 and made its structure more explicit, in particular by spelling out the interaction case in detail and by making clear the key invariants on the extended state S_+ used in the correspondence argument. To keep the exposition readable, we do not enumerate every PRISM semantic step.

Changes in the manuscript. Proof of Theorem 1 rewritten and expanded and interaction case detailed.

Reviewer comment. I have also some concerns about the way in which the choreography state is projected on different participants.

Changes in the manuscript. Definitions 4 (CTMC) and 5 (DTMC) have been updated. Now states are pre-generated making it more clear how the projection works.

Reviewer comment. When giving examples, the use of prime variables is not clear at this point.

Changes in the manuscript. We added a brief clarification in the manuscript.

Reviewer comment. I did not understand the explanation of foreach. What is the role of k ? What is op ? A formal encoding would be welcome.

Response. We apologise for the limited explanation of this construct. The basic idea is to provide an operator that quantifies over indices via the predicate $op(i)$. The formal translation is straightforward: we add one update for each role, assuming that the relevant roles can be determined statically.

Changes in the manuscript. We have updated the text where the foreach construct is discussed, with a more precise explanation of what it does.

Reviewer comment. I am puzzled about the notion of Non-deterministic Synchronization. [...] I have a couple of questions here:

1. Does the specification determine the direction of communication? In the provided example, it seems that the interaction could also be expressed in the opposite direction, i.e., $q \rightarrow p$. Is the direction of the interaction relevant, or is it never relevant?
2. Does any synchronization specification yield a correct choreography? Should there not be a requirement for some well-formedness condition on the `allsync` constructor?
3. Could you provide a formal definition of the encoding?

Response. The terminology we used is incorrect, thanks for pointing this out. `allsync` is not "nondeterministic" but it is still a probabilistic choice. We have rewritten (and expanded) the entire paragraph with a clearer explanation. As for the detailed questions:

1. in our choreography language, writing $q \longrightarrow \{p\}$ or $p \longrightarrow \{q\}$ is the same. However, it has an impact on the projection, since we need to distinguish one of the modules (the one on the left) because it has a different projection.
2. The notion of strong connectedness can be extended to `allsync`, as it is, after all, an interaction between the modules p_i . However, this is not necessary if we first translate it into standard interactions (and if-then-else constructs), since strong connectedness can then be checked at that level. In the new example we have provided, strong connectedness does not hold because the nested if-then-else constructs are not connected. As this part is not a formal development but rather a section justifying how our implementation extends the theoretical framework, we do not consider it necessary to present a connected choreography here. In any case (as noted in a comment added to the projection section), these choreographies can be transformed into connected ones by introducing additional interactions.
3. the translation should be straightforward once there is a precisely defined language for guards. See new text added.

Changes in the manuscript. The paragraph has been rewritten.

Reviewer comment. Example 3: The modules in this example share variables (y and x). I wonder whether there is a choreography that describes their behavior, since my understanding is that variables are local to modules in choreographies

Response. In the example, x and y are indeed owned by two different modules and are therefore shared only in read mode. In PRISM, all modules can read all variables, but only the owning module can modify its own variable. This is consistent with our choreographic reading, where updates are only performed as part of an interaction, so the write is still attributed to a specific role in that interaction. The only caveat is that, in choreographies, guards in conditionals must be local (i.e., evaluated by a single role). Thus, for this example, we could simulate the interactive execution (sync on a) by nesting two if-then-else constructs. However, since choreographies focus only on interactions, the internal actions (no label) cannot be simulated.

Reviewer comment. The choice of annotating choreographies with labels seems somewhat ad hoc. Why are they not introduced as a syntactic element [...] given that they are necessary and play a role in the implementation?

Response. It is certainly true that labels are a key aspect of the implementation. However, the purpose of this work is to define a higher-level language that abstracts away from such implementation details. Therefore, labels are part of the projection algorithm and are irrelevant to the programmer who writes a choreography. Nevertheless, we now use an intermediate choreographic language and a function for annotating choreographies in a “safe” way.

Reviewer comment. Projection of conditionals: I do not understand why the conditional is not projected.

Response. We believe this is a misunderstanding, and we are grateful for the opportunity to clarify it. The conditional *is* projected, but only in the module of the evaluating role p . This is consistent with the intended semantics: the guard E is evaluated by p and the result deterministically selects the branch. The point that may be confusing is that we do *not* require E to be local to p . As stated on page 5, E may mention variables owned by other modules; in PRISM, all modules can read all variables, while only the owning module can modify them. Therefore, the projection places the conditional in p ’s module, and the guard can still refer to remote variables, exactly as in the PRISM syntax.

Reviewer comment. Projection of choreography variable X : This relies on $\text{defs}(X)$. [...]

Response. We agree with the reviewer.

Changes in the manuscript. In the revised formulation, counters are replaced by explicit labels on choreographies, which makes the treatment of variables cleaner and more uniform. We have also expanded the proof for this case and included more details to make the reasoning fully explicit.

Reviewer comment. Projection of updates: Are you assuming that updates at the choreography level are well-formed in some way? For instance, could an assignment $y' = x$ occur where y belongs to p and x to q ? If not, this restriction should be stated clearly. Moreover, from which element of the choreography is the assignment derived? If I have missed this, it should nonetheless be clarified explicitly in the paper.

Response. As discussed above, assignments such as $y' = x$ are permitted even when x is located at p and y is located at q . At the choreography level, the restriction is simply that, in an interaction $p \rightarrow \{p_1, \dots, p_n\} : \Sigma_{j \in J} \lambda_j : u_j; C_j$, every update in each u_j assigns to a variable owned by one of $p, p_1, \dots, p_n$.

Changes in the manuscript. We have added a clear assumption on page 6 (new PDF).

Reviewer comment. Definition 4: Is it not the case that the choreography is annotated?

Response. Interactions are now also annotated. See new version.

Reviewer comment. - How are variables quantified? Are S , S' , and ι universally quantified?

Response. Yes, although now ι has disappeared.

Reviewer comment. The correspondence theorem (Theorem 1) appears somewhat weak [...]

Response. See our reply about your first comment.

Reviewer comment. Although I can follow the reasoning, I wonder what the fundamental limitation is here.

Changes in the manuscript. Added explanation on page 24.

Reviewer 2

Reviewer comment. The main changes as stated by the authors at the end of the Introduction are: [...] The paper does not help the reader in identifying them.

Response. Here is a list of the major differences between this latest version and the conference version (random order):

- presentation of three different macros that are used in the implementation
- a new, cleaner, projection function that no longer uses natural numbers for the control state
- proof of Theorem 1
- new case study: Simple Peer-To-Peer Protocol
- new case study: Synchronous Leader Election
- new case study: Dining Cryptographers
- new case study: Code Size Comparison
- more detailed discussions and related work

Reviewer comment. "The introduction should be more clear about which kind of concurrent process is amenable for choreographies, and for which class of concurrent processes choreographies are not suitable." The comment still applies for the new introduction.

Changes in the manuscript. We have added a paragraph in the introduction.

Reviewer comment. The experimental analysis in Section 7 needs to be improved.

Changes in the manuscript. This part has been updated accordingly.

Reviewer comment. Section 6 is irrelevant and can be removed.

Changes in the manuscript. Removed.

Reviewer comment. It is confusing that you use $\lambda_{1,2}$ as label in page 2 [...]

Changes in the manuscript. Fixed by rewriting the whole example paragraph in the introduction.

Reviewer comment. [...] the text uses sometimes the math symbols and sometimes the full name of the greek letters λ , μ , θ . [...]

Changes in the manuscript. Fixed.

Reviewer comment. I don't know if this intentional, but if one wants to specify no update, one has to write $x' = x$ (a void update). In PRISM one can simply write `true`, if I recall properly. This is related also to the syntax of PRISM introduced in 4.1.

Response. From a theoretical point of view, there is no difference, therefore we will leave this as it is. Our implementation allows also `True`.

Reviewer comment. Are you sure Def. 1 works as intended? What happens if x appears in u ? For example, $[x' = 0, y' = x]$. In Prism, y will be updated to the value of x in S (see remark at the bottom of <https://www.prismmodelchecker.org/manual/ThePRISMLanguage/Commands>), but in your case it will be set to the new value of x (0). This is also relevant to the definition of μ in page 9 and needs to be fixed.

Response. Yes sorry, our definition is incorrect.

Changes in the manuscript. We have now updated the definition so that it handles multiple updates at the same time.

Reviewer comment. General remark: the way you define unique identifiers for the control points in the choreographies is not self-explainable. Readers familiar with building control flow graphs and similar graphical structures from code will get it, but other readers may find it cryptic. I would suggest enriching the explanation of how control points sq are correctly created.

Changes in the manuscript. Added some text.

Reviewer comment. "the projected update $uj \downarrow q$ acts as a filter on the list of updates in uj " is this defined in the paper?

Changes in the manuscript. We have added further comments on this.

Reviewer 3

Reviewer comment. The pros and cons of choreographic programming as opposed to process-oriented programming are not elaborated upon very much.

Changes in the manuscript. Added discussion on pages 2 and 25–26.

Reviewer comment. How does this relate to the notions of implementability and realisability [...]?

Changes in the manuscript. We have extended related work on page 25.

Reviewer comment. Could the authors imagine a more thorough comparison, e.g., bisimilarity?

Changes in the manuscript. We have added a discussion at the end of Section 5.